

DeepSeek-V4 论文详细分析报告

论文标题: DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence
机构: DeepSeek-AI
发布日期: 2026年

目录

- [概述与核心贡献](#)
- [模型架构详解](#)
- [基础设施优化](#)
- [预训练](#)
- [后训练](#)
- [评测结果](#)
- [工程实践亮点](#)
- [局限性与未来方向](#)
- [总结与展望](#)

1. 概述与核心贡献

1.1 研究动机

随着推理模型（如 DeepSeek-R1、OpenAI o1）的兴起，测试时扩展（test-time scaling）成为提升 LLM 性能的新范式。然而，传统注意力机制的二次计算复杂度在处理超长上下文时成为瓶颈，限制了测试时扩展和长期任务（long-horizon tasks）的发展。

DeepSeek-V4 的目标是打破超长上下文处理的效率壁垒，使百万级 token 上下文窗口成为实用可行的标准配置。

1.2 模型家族

DeepSeek-V4 系列包含两个基于 MoE 架构的预览版本模型：

特性	DeepSeek-V4-Pro	DeepSeek-V4-Flash
总参数量	1.6T	284B
激活参数量	49B	13B
上下文窗口	1M tokens	1M tokens
预训练数据量	33T tokens	32T tokens
Transformer 层数	61	43
隐藏维度	7168	4096

1.3 核心创新

- 1. 混合注意力架构：结合 Compressed Sparse Attention (CSA) 和 Heavily Compressed Attention (HCA)
- 2. 流形约束超连接 (mHC)：增强传统残差连接
- 3. Muon 优化器：替代 AdamW，实现更快收敛和更高训练稳定性
- 4. FP4 量化感知训练：用于 MoE 专家权重和索引器 QK 路径

1.4 效率突破

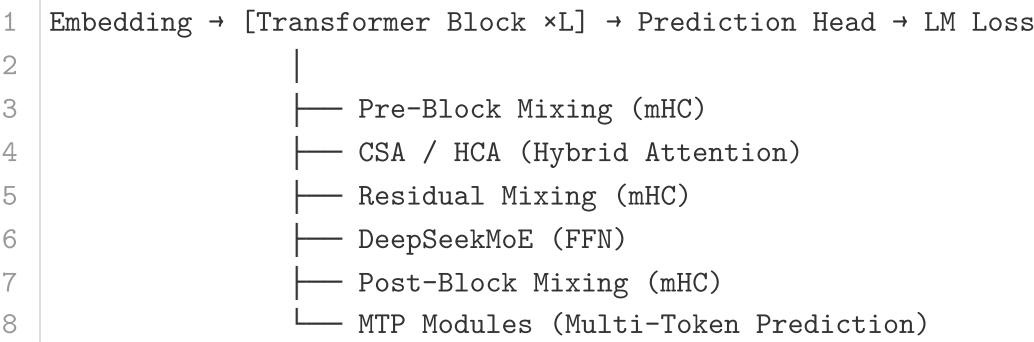
在 1M-token 上下文场景下（与 DeepSeek-V3.2 对比）：

指标	DeepSeek-V4-Pro	DeepSeek-V4-Flash
单 token 推理 FLOPs	27%	10%
KV 缓存大小	10%	7%

相较于 BF16 GQA8（头部维度 128）的基线配置，V4 的 KV 缓存降低了约 98%。

2. 模型架构详解

整体架构概述如下图所示——DeepSeek-V4 在保留 Transformer 主体和 MTP 模块的同时，引入 mHC、CSA/HCA 混合注意力、Muon 优化器三大升级：



2.1 继承自 V3 的设计

2.1.1 DeepSeekMoE：专家混合的架构演进

DeepSeek-V4 沿用 DeepSeekMoE 范式——设置细粒度路由专家（**Routed Experts**）和共享专家（**Shared Experts**）。每个 MoE 层由 1 个共享专家 + 多个路由专家组成，每个 token 激活其中 6 个路由专家。相较 V3 有以下关键改动：

(a) 激活函数变更

计算亲和度分数（affinity scores）的激活函数从 $\text{Sigmoid}(\cdot)$ 改为 $\text{Sqrt}(\text{Softplus}(\cdot))$ ：

$$\text{Sqrt}(\text{Softplus}(x)) = \sqrt{\ln(1 + e^x)}$$

这一改动带来的好处是：Softplus 相比 Sigmoid 在正值区域保持接近线性的响应，而 Sigmoid 在较大正值时趋于饱和，导数趋近于零。Softplus 能更细粒度地区分高亲和度专家之间的差异。外层 sqrt 起到压缩动态范围的作用，避免亲和度分数方差过大导致路由过于集中。

(b) 移除路由目标节点数约束

DeepSeek-V3 曾设置每组专家可路由到的目标节点数量上限以简化并行策略。V4 移除了这一约束——这意味着每个 token 理论上可以路由到任意专家组合。为了实现这一点，团队重新设计了并行策略，在训练框架中利用细粒度 EP（见 3.1 节）来维持效率。

(c) Hash Routing 替代前几层的 Dense FFN

V3 的前几层使用 Dense FFN（无专家路由的全连接层）。V4 将其替换为采用 **Hash Routing** 的 MoE 层（V4-Flash 前 3 层，V4-Pro 前 3 层）。Hash 路由策略根据输入 token ID 的预定义哈希函数直接确定目标专家：

$$\text{Expert}(t) = \text{Hash}(\text{TokenID}(t)) \bmod N_{\text{experts}}$$

这完全消除了路由网络的计算开销和负载均衡需求——哈希函数天然保证均匀分布。由于模型底层主要学习语言的表层特征（如词法、语法模式），这些特征与 token 身份强相关，Hash 路由足以覆盖。同时，由于路由与输入上下文无关，这一层的输出完全确定，稳定性更强。

(d) 负载均衡策略

沿用辅助损失自由（auxiliary-loss-free）策略——通过动态调整每个专家的偏置项来平衡负载，不引入额外的训练目标。此外增加了一个微小的序列级均衡损失（权重 0.0001），防止单个序列内的极端不均衡：

$$\mathcal{L}_{\text{balance}} = 0.0001 \cdot \frac{1}{L} \sum_{l=1}^L \text{Var}(\text{Count}_{l,1}, \dots, \text{Count}_{l,N_{\text{exp}}})$$

其中 $\text{Count}_{l,e}$ 是第 l 层第 e 个专家在当前序列中被选中的次数。偏置更新速度设为 0.001。

模型配置对比：

MoE 配置	V4-Flash	V4-Pro
路由专家数	256	384
共享专家数	1	1
每 token 激活专家数	6	6
专家中间隐藏维度	2048	3072

2.1.2 Multi-Token Prediction (MTP)

MTP 模块的配置与 DeepSeek-V3 完全一致。MTP 在模型顶部添加额外的预测模块，自回归地预测后续 D 个 token ($D = 1$)，其目标是鼓励模型学习更长程的依赖关系。MTP 损失权重在训练大部分阶段为 0.3，在学习率衰减开始时降至 0.1。

2.2 流形约束超连接 (mHC)

2.2.1 问题背景：标准 HC 的不稳定性

传统残差连接 (ResNet-style) 的形式为 $X_{l+1} = X_l + \mathcal{F}_l(X_l)$ ，其中残差流宽度等于隐藏维度 d 。标准 Hyper-Connections (HC) 将残差流宽度扩展为 n_{hc} 倍：

残差状态 $X_l \in \mathbb{R}^{n_{hc} \times d}$ (n_{hc} 个 d 维向量)。HC 引入三个线性映射：

- 输入映射** $A_l \in \mathbb{R}^{1 \times n_{hc}}$ ：将 n_{hc} 个残差流组合为层输入
- 残差变换** $B_l \in \mathbb{R}^{n_{hc} \times n_{hc}}$ ：在各残差流之间混合信息
- 输出映射** $C_l \in \mathbb{R}^{n_{hc} \times 1}$ ：将层输出投影回残差流

更新公式：

$$X_{l+1} = B_l X_l + C_l \cdot \mathcal{F}_l(A_l X_l) \quad (1)$$

HC 解耦了残差宽度和隐藏维度——由于 $n_{hc} \ll d$ (V4 中 $n_{hc} = 4$)，额外计算开销极小但提供了一条互补的扩展轴。然而，团队发现：当堆叠多层时，标准 HC 频繁出现数值不稳定性，阻碍了 HC 的规模化应用。

2.2.2 mHC 核心创新：双随机矩阵流形约束

mHC 的关键洞察是将残差映射矩阵 B_l 约束到 Birkhoff 多面体 (双随机矩阵流形)：

$$\mathcal{M} := \{M \in \mathbb{R}^{n \times n} \mid M \mathbf{1}_n = \mathbf{1}_n, \mathbf{1}_n^T M = \mathbf{1}_n^T, M \geq 0\} \quad (2)$$

约束的数学意义：

- 谱范数有界** (Perron-Frobenius 定理)：双随机矩阵的谱范数 $\|B_l\|_2 \leq 1$ ，等号仅当 B_l

为置换矩阵时成立。这保证了残差变换是**非扩张**（non-expansive）的——在正向传播中，残差信号的能量不会逐层放大；在反向传播中，梯度不会被指数级放大。

2. **乘法封闭性**：双随机矩阵的乘积仍是双随机矩阵（ $\mathcal{M}$ 在半群运算下封闭）。这意味着 $\|B_{l_2}B_{l_1}\|_2 \leq 1$ 对任意层组合都成立，保证深层堆叠时的全局稳定性。

3. **信号保持性**： $\mathbf{1}_n^T B_l = \mathbf{1}_n^T$ 意味着残差流的总和（宏观信号）被精确保持，避免了标准 HC 中可能出现的信号漂移或消失。

同时， A_l 和 C_l 通过 Sigmoid 函数约束为非负且有界（ C_l 倍乘 2 以允许稍大的动态范围）：

$$A_l = \sigma(\tilde{A}_l), \quad C_l = 2\sigma(\tilde{C}_l) \quad (3)$$

非负性约束避免了信号在不同残差流之间相互抵消的风险（符号翻转导致的信号湮灭）。

2.2.3 动态参数化与约束实施

mHC 的三个映射参数均采用**动态+静态混合参数化**：

步骤 1：输入归一化

首先将残差状态展开并归一化：

$$\hat{X}_l = \text{RMSNorm}(\text{vec}(X_l)) \in \mathbb{R}^{1 \times n_{hc}d} \quad (4)$$

步骤 2：生成原始参数

$$\tilde{A}_l = \alpha_l^{\text{pre}} \cdot (\hat{X}_l W_l^{\text{pre}}) + S_l^{\text{pre}} \quad (5)$$

$$\tilde{B}_l = \alpha_l^{\text{res}} \cdot \text{Mat}(\hat{X}_l W_l^{\text{res}}) + S_l^{\text{res}} \quad (6)$$

$$\tilde{C}_l = \alpha_l^{\text{post}} \cdot (\hat{X}_l W_l^{\text{post}})^T + S_l^{\text{post}} \quad (7)$$

其中：

- $W_l^{\text{pre}}, W_l^{\text{post}} \in \mathbb{R}^{n_{hc}d \times n_{hc}}$ 和 $W_l^{\text{res}} \in \mathbb{R}^{n_{hc}d \times n_{hc}^2}$ 为可学习的动态分量权重
- $S_l^{\text{pre}}, S_l^{\text{post}}, S_l^{\text{res}}$ 为可学习的静态偏置
- $\alpha_l^{\text{pre}}, \alpha_l^{\text{res}}, \alpha_l^{\text{post}}$ 为初始化为小值的可学习门控因子
- $\text{Mat}(\cdot)$ 将 $\mathbb{R}^{1 \times n_{hc}^2}$ 向量重塑为 $\mathbb{R}^{n_{hc} \times n_{hc}}$ 矩阵

门控因子初始化为小值的意义在于：训练初期 mHC 接近恒等映射（ $B_l \approx I, C_l \approx 0$ ），使模型平滑地从行为接近传统残差连接的状态开始学习。

步骤 3：施加约束

对 $\tilde{A}_l$ 和 $\tilde{C}_l$ 直接通过 Sigmoid 约束（公式 3）。对 $\tilde{B}_l$ ，通过 Sinkhorn-Knopp 算法将其投影到双随机矩阵流形：

$$M^{(0)} = \exp(\tilde{B}_l) \quad (\text{保证正性}) \quad (8)$$

$$M^{(t)} = \mathcal{T}_r(\mathcal{T}_c(M^{(t-1)})) \quad (\text{交替行列归一化}) \quad (9)$$

$$B_l = M^{(t_{\max})}, \quad t_{\max} = 20 \quad (10)$$

Sinkhorn-Knopp 算法（行列交替归一化）在数学上收敛到给定正矩阵的唯一双随机投影。20 次迭代在实践中有充分的收敛性——由于 $n_{hc} = 4$ 很小，每次迭代仅涉及 4×4 矩阵的行/列归一化，计算开销可忽略。

V4 的 mHC 配置： $n_{hc} = 4, t_{\max} = 20$ 。

2.2.4 mHC 与标准残差/标准 HC 的对比分析

特性	标准残差连接	标准 HC	mHC (DeepSeek-V4)
残差流宽度	d	$n_{hc} \times d$	$n_{hc} \times d = 4d$
信号变换	恒等	无约束 B_l	$\ B_l\ _2 \leq 1$
深层稳定性	梯度消失/爆炸风险	频繁不稳定	保证非扩张
表达能力	基准	增加	增加且约束正则化
额外参数	0	$\sim O(n_{hc}^2)$	$\sim O(n_{hc}^2 d + n_{hc}^2)$
总 wall-time 开销	0	低	6.7%（经优化后）

2.3 混合注意力机制

这是 DeepSeek-V4 **最重要的**架构创新。核心思想：用两种不同策略的注意力层交织排列，在计算效率和模型质量之间取得最优平衡。

在 V4 中，前两层使用纯滑动窗口注意力（V4-Flash）或 HCA（V4-Pro），后续层中 CSA 和 HCA 以**交织（interleaved）**方式交替出现。

2.3.1 Compressed Sparse Attention (CSA)

CSA 的策略可概括为"**先压缩，再稀疏选择，最后精确计算**"。

阶段一：重叠压缩 KV 缓存

设输入隐藏状态序列 $H \in \mathbb{R}^{n \times d}$ （ n 序列长度， d 隐藏维度）。

(1) 计算两组 KV 项 $C^a, C^b \in \mathbb{R}^{n \times c}$ 及其压缩权重 $Z^a, Z^b \in \mathbb{R}^{n \times c}$:

$$\begin{aligned} C^a &= H \cdot W^{aKV}, & Z^a &= H \cdot W^{aZ} \\ C^b &= H \cdot W^{bKV}, & Z^b &= H \cdot W^{bZ} \end{aligned} \quad (11)$$

其中 $W^{aKV}, W^{bKV}, W^{aZ}, W^{bZ} \in \mathbb{R}^{d \times c}$ ， c 为注意力头维度（V4 中 $c = 512$ ）。

(2) **重叠压缩**：对于每个压缩块 i ，融合 m 个 C^a （区间 $[mi, m(i+1) - 1]$ ）及其相邻的前 m 个 C^b （区间 $[m(i-1), mi - 1]$ ）。首先计算 Softmax 权重：

$$[S_{mi:m(i+1)-1}^a; S_{m(i-1):mi-1}^b] = \text{Softmax}_{\text{row}}([Z_{mi:m(i+1)-1}^a + B^a; Z_{m(i-1):mi-1}^b + B^b]) \quad (12)$$

其中 $B^a, B^b \in \mathbb{R}^{m \times c}$ 为可学习的位置偏置。然后加权求和得到压缩 KV 项：

$$C_i^{\text{Comp}} = \underbrace{\sum_{j=mi}^{m(i+1)-1} S_j^a \odot C_j^a}_{\text{当前块的 } C^a} + \underbrace{\sum_{j=m(i-1)}^{mi-1} S_j^b \odot C_j^b}_{\text{前邻块的 } C^b} \quad (13)$$

这一设计的精巧之处：

- 每个 C_i^{Comp} 使用了 $2m$ 个原始 KV 项，但相邻块之间有 m 个重叠——因此 C_i 中的 C^b 部分同时也是 C_{i-1} 中的 C^a 部分
- 净压缩率为 $1/m$ （序列长度从 n 压缩到 n/m ）
- 重叠设计创建了跨块的信息桥梁，缓解压缩造成的块边界信息断裂

V4 中 CSA 压缩率 $m = 4$ ，即每 4 个 token 压缩为 1 个 KV 项。

阶段二：Lightning Indexer 稀疏选择

在获得压缩 KV $C^{\text{Comp}} \in \mathbb{R}^{n/m \times c}$ 后，类似地计算压缩索引器 keys $K^{\text{IComp}} \in \mathbb{R}^{n/m \times c^I}$ （ $c^I = 128$ 为索引器头维度）。

对于查询 token t ，首先生成**低秩索引器查询**：

$$c_t^Q = h_t \cdot W^{DQ}, \quad W^{DQ} \in \mathbb{R}^{d \times d_c} \quad (14)$$

$$[q_t^{I,1}; \dots; q_t^{I,n_h^I}] = q_t^I = c_t^Q \cdot W^{IUQ}, \quad W^{IUQ} \in \mathbb{R}^{d_c \times c^I n_h^I} \quad (15)$$

其中 d_c 为查询压缩维度， $n_h^I = 64$ 为索引器查询头数。低秩设计的动机：在生成长度为 n/m 的索引器查询向量时，先降维到 d_c 再上投影比直接使用全维度（ d ）更高效。

然后计算每个索引器头的权重和索引分数：

$$[w_t^{I,1}; \dots; w_t^{I,n_h^I}] = w_t^I = h_t \cdot W^w, \quad W^w \in \mathbb{R}^{d \times n_h^I} \quad (16)$$

$$\mathbf{I}_{t,s} = \sum_{h=1}^{n_h^I} w_t^{I,h} \cdot \text{ReLU}(q_t^{I,h} \cdot K_s^{\text{IComp}}) \quad (17)$$

ReLU 的使用意味着：只有查询-键正相关的压缩块才贡献索引分数，负相关被截断为零——这与标准注意力中所有键都参与 softmax 有本质不同，提供了一种硬过滤机制。

根据 $\mathbf{I}_{t,:}$ 进行 Top-k 选择（V4-Pro $k = 1024$ ，V4-Flash $k = 512$ ），保留得分最高的 k 个压缩 KV 项：

$$C_t^{\text{SprsComp}} = \{C_s^{\text{Comp}} \mid \mathbf{I}_{t,s} \in \text{Top-}k(\mathbf{I}_{t,:})\} \quad (18)$$

阶段三：Shared KV Multi-Query Attention

稀疏选择后的压缩 KV 项以 **MQA** 方式参与核心注意力——即压缩 KV 同时充当 key 和 value（这是可行的，因为 KV 来自同一压缩过程）。查询向量同样来自低秩隐向量：

$$[q_{t,1}; \dots; q_{t,n_h}] = q_t = c_t^Q \cdot W^{UQ}, \quad W^{UQ} \in \mathbb{R}^{d_c \times cn_h} \quad (19)$$

$$o_{t,i} = \text{CoreAttn} \left(\text{query} = q_{t,i}, \text{key} = C_t^{\text{SprsComp}}, \text{value} = C_t^{\text{SprsComp}} \right) \quad (20)$$

V4 配置： $n_h = 64$ (Flash) / 128 (Pro), $c = 512$, $d_c = 1024$ (Flash) / 1536 (Pro)。

阶段四：分组输出投影

直接投影 ($n_h \times c$) 维的输出到 d 维 (Pro: $128 \times 512 = 65536 \rightarrow 7168$) 计算量过大。采用**分组投影策略**：

将 n_h 个输出分为 g 组 (Flash $g = 8$, Pro $g = 16$)，每组先投影到中间维度 $d_g = 1024$ ：

$$o_{t,i}^{G'} = o_{t,i}^G \cdot W_i^G, \quad W_i^G \in \mathbb{R}^{(cn_h/g) \times d_g} \quad (21)$$

$$\hat{o}_t = [o_{t,1}^{G'}; \dots; o_{t,g}^{G'}] \cdot W^O, \quad W^O \in \mathbb{R}^{(d_g g) \times d} \quad (22)$$

分组投影的参数量约为 $g \cdot \frac{cn_h}{g} \cdot d_g + d_g g \cdot d$ ，远小于直接投影的 $cn_h \cdot d_o$ 。这带来约 2-3 $\times$ 的投影阶段计算量缩减。

2.3.2 Heavily Compressed Attention (HCA)

HCA 走极端压缩路线：大幅压缩但不做稀疏选择。

压缩阶段（无重叠，压缩率 $m' = 128$ ）：

$$C = H \cdot W^{KV}, \quad Z = H \cdot W^Z, \quad W^{KV}, W^Z \in \mathbb{R}^{d \times c} \quad (23)$$

$$S_{m':m'(i+1)-1} = \text{Softmax}_{\text{row}}(Z_{m':m'(i+1)-1} + B), \quad B \in \mathbb{R}^{m' \times c} \quad (24)$$

$$C_i^{\text{Comp}} = \sum_{j=m'i}^{m'(i+1)-1} S_j \odot C_j \quad (25)$$

HCA 将序列长度从 n 直接压缩到 $n/128$ 。因为压缩后的 KV 项已非常少，不需要稀疏选择——直接 Dense Attention 的计算量也完全可控。

注意力阶段：与 CSA 共享同一套 MQA + 分组投影机制。

2.3.3 辅助技术细节

(a) 滑动窗口注意力分支

CSA 和 HCA 都因压缩/稀疏性而无法访问同一压缩块内的其他 token。为解决这一问题，额外添加一个滑动窗口注意力分支——每个查询 token 额外关注最近的 $n_{win} = 128$ 个**未压缩** token。在核心注意力中，这些滑动窗口 KV 项与压缩 KV 项拼接后一起参与计算。滑动窗口的大小经过精心选择——足以覆盖绝大多数局部依赖（如相邻句子、代码块内的引用），同时增量

存储开销很小。

(b) Query/KV 项归一化

在核心注意力操作之前，对每个注意力头的查询向量和压缩 KV 项执行额外的 RMSNorm：

$$\tilde{q}_{t,i} = \text{RMSNorm}(q_{t,i}), \quad \tilde{K}^{\text{Comp}} = \text{RMSNorm}(K^{\text{Comp}}) \quad (26)$$

归一化避免了注意力 logits 因压缩过程中向量范数的累积而爆炸——在 1M 上下文场景下，不归一化的 logits 可能达到使 softmax 溢出 FP16 范围的程度。这也使得 Muon 优化器无需 QK-Clip 技术（见 2.4 节）。

(c) 部分 Rotary Positional Embedding (RoPE)

仅对查询和 KV 向量的**最后 64 维**应用 RoPE。这种"部分 RoPE"设计在保留位置感知能力的同时，降低了需要高精度存储的维度数量（见混合精度存储）。

一个独特的处理：因为压缩 KV 同时作为 key 和 value (MQA)，核心注意力的输出 $o_{t,i}$ 携带绝对位置编码。团队对每个 $o_{t,i}$ 的最后 64 维**反向应用 RoPE**（位置 $-\text{id}(t)$ ），使得每个压缩 KV 块对输出的贡献与它和查询的相对距离相关，实现相对位置编码效果。

(d) Attention Sink

引入可学习的 sink logits $\{z'_1, \dots, z'_{n_h}\}$ 。修改注意力分数计算：

$$s_{h,i,j} = \frac{\exp(z_{h,i,j})}{\sum_k \exp(z_{h,i,k}) + \exp(z'_h)} \quad (27)$$

$\exp(z'_h)$ 作为分母中的一个常数偏移量——相当于虚拟了一个全零的"注意力沉没令牌"。这让每个注意力头可以灵活地"排出"一部分注意力总分，使有效注意力分值不等于 1（甚至接近 0）。在长上下文中，大量不相关的压缩块可以有效地被"沉没"，从而增强真正相关块的注意力权重。

2.3.4 模型配置总览

注意力配置	DeepSeek-V4-Flash	DeepSeek-V4-Pro
Transformer 层数	43	61
前两层类型	纯滑动窗口	HCA
CSA 压缩率 m	4	4
HCA 压缩率 m'	128	128
查询头数 n_h	64	128
头维度 c	512	512
查询压缩维度 d_c	1024	1536
索引器查询头数 n_h^I	64	64
索引器头维度 c^I	128	128
Attention top-k	512	1024
投影组数 g	8	16
中间投影维度 d_g	1024	1024
滑动窗口 n_{win}	128	128

2.3.5 效率量化分析

与 BP16 GQA8（头维 128）基线对比：

- **KV 缓存**：CSA 层压缩 $4\times$ + HCA 层压缩 $128\times$ + 混合精度（RoPE dims BF16 / 其余 FP8 减半）+ 更少的注意力层。综合效果：1M 上下文时 KV 缓存约为基线的 2%。
- **注意力 FLOPs**：CSA 层的 DSA 仅计算 k 个压缩 KV 项（而非全部 n 个），HCA 层仅计算 $n/128$ 个压缩项。FP4 精度索引器计算进一步降低开销。

与 DeepSeek-V3.2 对比（已在 1M 上下文下较为高效）：

效率指标	V4-Flash	V4-Pro
单 token 推理 FLOPs（等效 FP8）	10%	27%
KV 缓存大小	7%	10%

2.4 Muon 优化器

2.4.1 基本原理

Muon 与传统 Adam/AdamW 的本质区别在于参数更新的计算方式。Adam 系列是**逐元素**优化器——每个参数独立维护一阶/二阶动量。Muon 则是**逐矩阵**优化器——对每个逻辑独立的权重矩阵整体计算更新。

```

1 Algorithm: Muon Optimization for DeepSeek-V4
2
3 Input:  $\eta$  (learning rate),  $\mu$  (momentum),  $\lambda$  (weight decay),  $\gamma$  (RMS rescale)
4 For each step  $t$ :
5   For each logically independent weight  $W \in \mathbb{R}^{n \times m}$ :
6     1.  $G_t = \nabla_W L_t(W_{t-1})$  // Gradient
7     2.  $M_t = \mu \cdot M_{t-1} + G_t$  // Momentum
8     3.  $O_t' = \text{HybridNewtonSchulz}(\mu \cdot M_t + G_t)$  // Nesterov +
Orthogonalize
9     4.  $O_t = O_t' \cdot \sqrt{\max(n, m)} \cdot \gamma$  // Rescale RMS
10    5.  $W_t = W_{t-1} \cdot (1 - \eta \cdot \lambda) - \eta \cdot O_t$  // Weight decay +
Update

```

步骤解析:

- 动量累积** (步骤 2): 与 AdamW 相同, 使用指数移动平均平滑梯度。
- Nesterov 技巧** (步骤 3 中的 $\mu M_t + G_t$): 先沿动量方向"外推"一步, 再在该位置计算梯度。相比标准动量, Nesterov 加速梯度具有更快的理论收敛速度。
- Newton-Schulz 正交化** (步骤 3): Muon 的核心操作——将梯度矩阵近似正交化, 使其奇异值接近 1。这等价于去除梯度在不同方向上的尺度差异, 使更新"各向同性"。
- RMS 缩放** (步骤 4): 将正交化后的更新矩阵的 RMS 值缩放到 $\sqrt{\max(n, m)} \times \gamma$ 。 γ 设置为 0.18 以复用 AdamW 的学习率超参数——这使得 Muon 和 AdamW 可以在同一训练中共享学习率调度。
- 权重衰减** (步骤 5): 与 AdamW 的 decoupled weight decay 一致。

2.4.2 Hybrid Newton-Schulz 迭代详解

Newton-Schulz 迭代的目标: 对归一化后的矩阵 $M_0 = M/\|M\|_F$, 通过迭代逼近其**极分解** UV^T (其中 $M = U\Sigma V^T$ 为 SVD):

$$M_k = aM_{k-1} + b(M_{k-1}M_{k-1}^T)M_{k-1} + c(M_{k-1}M_{k-1}^T)^2M_{k-1} \quad (28)$$

DeepSeek-V4 采用**两阶段混合策略**共 10 次迭代:

阶段	迭代次数	系数 (a, b, c)	目标
Stage 1	1-8	(3.4445, -4.7750, 2.0315)	快速驱动奇异值趋近 1
Stage 2	9-10	(2, -1.5, 0.5)	精确稳定奇异值为 1

为什么需要两阶段?

第一阶段的系数来自 5 阶 Householder 方法的展开, 收敛速度极快, 但可能在接近 1 时出现微小超调。第二阶段的系数来自标准的 3 阶展开, 在奇异值接近 1 时行为更稳定, 不会超调——它精确地将奇异值锚定在 1。比 Liu et al. (2025) 的纯 5 阶展开更稳定, 比纯 3 阶展开收敛更快。

Newton-Schulz 与 Adam 的类比：

在 Adam 中， $\hat{m}_t/\sqrt{\hat{v}_t}$ 可以理解为对梯度向量的逐元素"白化"——将每个维度的更新缩放到单位 RMS。Newton-Schulz 是对梯度矩阵的矩阵级"白化"——将矩阵的所有奇异值缩放到 1，使更新在矩阵的所有方向上均匀。

2.4.3 为什么 V4 可以不用 QK-Clip

Liu et al. (2025) 在使用 Muon 时采用了 QK-Clip 技术——限制注意力 logits 的大小，因为 Muon 的正交化更新可能导致注意力 logits 的范数逐渐增大。DeepSeek-V4 不采用 QK-Clip，原因在于其注意力架构允许直接对查询和 KV 执行 RMSNorm（公式 26）——这从根源上防止了 logits 爆炸，无需额外的裁剪操作。

2.4.4 适用范围

优化器	适用模块
Muon	注意力层权重、FFN/专家权重、MTP 模块权重、mHC 动态生成权重
AdamW	Embedding 层、预测头（LM Head）、mHC 的静态偏置 S_l 和门控 α_l 、所有 RMSNorm 权重

保留 AdamW 的模块有其原因：Embedding 和预测头通常共享权重，逐元素更新的 AdamW 在处理稀疏梯度（token 级）时更稳健。RMSNorm 的权重极小（仅为 d 维向量），矩阵级的 Muon 没有优势。

3. 基础设施优化

DeepSeek-V4 的架构创新带来了巨大的工程挑战。本节深入分析团队如何通过一系列系统级优化，使这些创新在训练和推理中高效运行。

3.1 细粒度通信-计算重叠的专家并行

3.1.1 问题分解

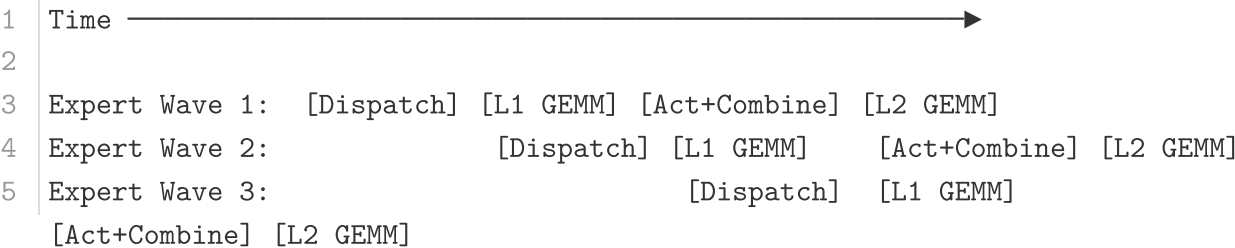
MoE 层可分解为四个子阶段：

1	Token Dispatch → Linear-1 GEMM → SwiGLU Act + FP8 Cast → Token Combine → Linear-2 GEMM			
2	(通信密集)	(计算密集)	(计算密集)	(通信密集)
	(计算密集)			

关键发现：**通信总时间小于计算总时间**。以 V4-Pro 为例，每个 token-expert 对需要 $6hd$ FLOPs 的计算（SwiGLU gate + up + down 投影），但仅需要 $3h$ bytes 的通信（FP8 Dispatch + BF16 Combine）。因此通信延迟理论上可被完全隐藏。

3.1.2 Wave 调度机制

核心策略是将专家划分为多个 **wave**，每个 wave 包含一小部分专家：



在稳态下，三个操作同时进行：

- 当前 wave 的 GEMM 计算
- 下一 wave 的 token dispatch (all-to-all)
- 已完成专家的结果 combine (all-to-all)

这形成一个细粒度的流水线，使计算和通信都保持连续。这一方案在**极端场景**（如 RL rollout 的小批次长尾）下效果最为显著，因为传统的粗粒度调度在小批次时通信占比会急剧上升。

3.1.3 性能数据

场景	加速比	平台
通用推理	1.50-1.73×	NVIDIA GPU / Huawei Ascend NPU
RL rollout / 高速 Agent 服务	≤ 1.96×	延迟敏感，小批次
理论最大（V4-Flash 配置，对比不重叠基线）	1.92×	—

3.1.4 硬件协同设计建议

论文基于 EP 开发经验提出了四项给硬件厂商的建议：

(a) 计算-通信比率（而非带宽）决定一切

通信可完全隐藏的充要条件：

$$\frac{C}{B} \leq \frac{V_{\text{comp}}}{V_{\text{comm}}}$$

对于 V4-Pro（SwiGLU: $6hd$ FLOPs, $3h$ bytes 通信）：

$$\frac{C}{B} \leq 2d = 6144 \text{ FLOPs/Byte}$$

这意味着每 GBps 互联带宽最多可支持 6.1 TFLOP/s 的计算。一旦带宽满足此阈值，继续增加带宽的收益递减——应将硅面积投入计算单元而非通信。

(b) 功耗预算

极端 kernel 融合同时驱动计算、内存、网络到高负载，功耗限制成为性能瓶颈。建议未来硬件为这种全并发工作负载提供充分的功耗裕量。

(c) 通信原语

团队采用 **pull-based** 方式（GPU 主动从远程 GPU 读取数据），避免了 fine-grained push 的高通知延迟。未来硬件若支持更低延迟的跨 GPU 信令，push 模式会更可行，通信模式也会更自然。

(d) 激活函数

建议用低成本的逐元素激活函数替代 SwiGLU：

- 去除指数/除法操作，减轻 GEMM 后处理负担
- 在同参数预算下，去除 gate 投影可扩大中间维度 d ，进一步降低带宽要求

3.1.5 开源实现

MegaMoE——CUDA mega-kernel 实现已作为 DeepGEMM 组件开源：<https://github.com/deepseek-ai/DeepGEMM/pull/304>

3.2 TileLang：兼顾生产力与性能的 Kernel 开发 DSL

DeepSeek-V4 的精巧架构在实际开发中会产生数百个细粒度的 **Torch ATen 算子**。直接手写 CUDA kernel 的开发效率无法跟上架构迭代速度。TileLang 作为 DSL 解决了这一矛盾——同一代码库中快速原型与深度优化共存。

3.2.1 Host Codegen：消除 CPU 侧调用瓶颈

随着加速器性能增长，CPU 侧的调度开销日益突出。传统方法在 Python 中进行运行时合约校验，每次 kernel 调用产生固定的 host 开销（数十至数百微秒）。对于 V4 中大量小而优化的 kernel，这种开销会严重影响 GPU 利用率。

方案：在 IR 级别共同生成设备 kernel 和轻量 host launcher：

1. 从前端解析数据类型、秩/形状约束、stride/layout 假设等元数据
2. 将 launcher 降低到基于 TVM-FFI 框架的**生成式 host 源代码**
3. 运行时由生成的 host 代码执行校验和参数 marshaling，所有 per-invocation 检查移出 Python 执行路径

效果：CPU 侧校验开销从数十微秒降至 1 微秒以下。

3.2.2 Z3 SMT 求解器辅助的形式化整数分析

TileLang kernel 中涉及复杂张量索引算术。在编译的布局推断、内存冲突检测、边界分析等 pass 中，编译器需要验证整数表达式是否满足特定属性以启用优化。

方案：将 Z3 SMT 求解器集成到 TileLang 的代数系统中：

- 将 TileLang 整数表达式翻译为 Z3 的 **QF_NIA**（无量词非线性整数算术）理论
- QF_NIA 基于整数线性规划求解器，无缝解决 kernel 中常见的标准线性整数表达式
- 固有的非线性推理能力可处理高级挑战（如可变张量形状的向量化）

性能：在合理资源限制下，Z3 提升整体优化性能，编译时开销仅数秒。影响覆盖向量化、barrier 插入、代码简化等多个 pass。

3.2.3 数值精度与位精确可重现性

精度策略：

- **默认禁用 fast-math**：精度影响操作仅作为显式 opt-in 前端算子提供（如 `T.__exp`，`T.__log`，`T.__sin`）
- **IEEE-754 支持**：需要严格 IEEE 语义时，提供带显式舍入模式的 IEEE 兼容内建函数（`T.ieee_fsqrt`，`T.ieee_fdiv`，`T.ieee_add`）
- **位精确验证**：TileLang 的代数简化和降低规则与主流 CUDA 工具链对齐，避免引入非预期的位级差异；Layout 标注允许固定 layout 相关的降低决策

设计哲学：保守默认不牺牲性能——在保守默认下 TileLang kernel 仍具有竞争力，同时暴露 knobs 允许选择性放宽数值约束以获得更高速度。

3.2.4 与 TileLang 社区的协作

团队与 TileLang 社区紧密协作，推动更灵活、高效、稳定的 kernel 开发工作流。这对于验证阶段的注意力变体快速原型至关重要。

3.3 批次不变性与确定性 Kernel 库

设计目标：端到端保证训练和推理之间的**位精确一致性**——对于调试、稳定性分析和后训练行为一致性至关重要。

3.3.1 批次不变性 (Batch Invariance)

批次不变性 = 任意 token 的输出在批次中的位置改变时保持位精确不变。

挑战 1：注意力解码

问题：传统 Flash-Decoding 使用 **split-KV** 方法——将单个序列的注意力计算分布到多个 SM 来平衡负载。但 split-KV 破坏了批次不变性（因为累加顺序取决于 SM 调度）。

不能用 split-KV 的后果：严重的 **wave 量化问题**——当序列长度不是 SM 数量 $\times$ chunk 大小的整数倍时，最后一个 wave 只填满部分 SM，GPU 利用率骤降。

V4 的双 kernel 策略：

Kernel	适用场景	策略
Kernel 1	完全占满的 wave	单 SM 处理完整序列→高吞吐
Kernel 2	最后的部分 wave	多 SM 并行处理→低延迟

核心技巧：Kernel 2 的累加顺序被精心设计为与 **Kernel 1 完全一致**。利用 **分布式共享内存**（thread-block cluster 特性）实现 SM 间高速数据交换。两个 kernel 的输出位精确相同。最终开销可忽略。

挑战 2：矩阵乘法

问题：cuBLAS 不能保证批次不变性。小批次场景中，split-k 技术常用于提升性能但也破坏不变性。

方案：端到端替换为 **DeepGEMM**。对于大多数场景放弃 split-k，通过自定义优化使性能匹配甚至超越标准 split-k。

3.3.2 确定性 (Determinism)

确定性 = 相同输入多次运行得到位精确相同的输出。非确定性通常源于浮点加法的不可结合性（`atomicAdd`）。

注意力反向：

标准实现使用 `atomicAdd` 向 KV token 累加梯度→非确定性。

方案：每个 SM 分配独立累加缓冲区，所有 SM 完成后执行**全局确定性求和**（使用树形归约确保固定累加顺序）。

MoE 反向：

多个 rank 的不同 SM 同时写入同一 rank 的共享接收缓冲区→写位置协商引入非确定性。

方案：

- 单 rank 内：token 顺序预处理机制，确定化发送数据顺序
- 多 rank 间：缓冲区隔离，不同 rank 写入不同缓冲区区域

mHC 矩阵乘法：

mHC 涉及输出维仅 24 的矩阵乘法。小批次时不得不用 split-k。方案：输出各 split 部分分开输出，在后续独立 kernel 中执行确定性归约。

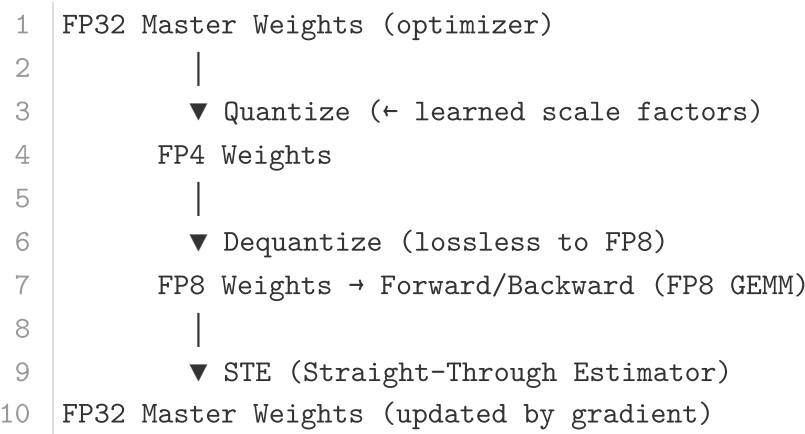
3.4 FP4 量化感知训练 (QAT)

3.4.1 量化目标与范围

组件	量化精度	目的
MoE 路由专家权重	FP4 (MXFP4)	GPU 内存消减（专家权重是最大内存占用源）
CSA 索引器 QK 路径	FP4	QK 激活以 FP4 缓存/加载/乘算，加速长上下文注意力
索引分数 $I_{:,i}$	FP32→BF16	Top-k 选择器 2× 加速

3.4.2 MoE 权重量化：FP4→FP8 无损反量化

QAT 流程：



为什么 FP4→FP8 反量化是无损的？

FP8 (E4M3) 格式有 2 个额外的指数位（相比 FP4 E2M1），提供更大的动态范围。具体机制：

- FP4 子块（ 1×32 tiles）有自己的缩放因子
- FP8 量化块（ 128×128 tiles）有更大的动态范围
- 只要同一 FP8 块内 FP4 子块的最大/最小缩放因子之比不超过 FP8 动态范围阈值，子块的精细尺度信息可完全被 FP8 的扩展动态范围吸收

团队实证验证了当前权重满足此条件。这意味着：整个 QAT 流水线**完全复用现有 FP8 训练框架，无需任何修改**。

反向传播时，梯度相对于前向传播中使用的 FP8 权重计算，通过 STE 直接传播到 FP32 master 权重——等效于"直接通过量化操作进行直通估计"。这避免了重新量化转置权重的需求。

3.4.3 推理与 RL Rollout 中的真实 FP4

在推理和 RL rollout（不涉及反向传播）阶段，直接使用**真实 FP4 量化权重**（而非模拟量化）：

- 模型行为与在线部署完全一致
- 减少 kernel 内存加载实现实际加速
- 显著降低内存消耗

3.4.4 Indexer QK 路径

索引器的 QK 激活以 FP4 精度缓存、加载和乘法运算。索引分数从 FP32 量化到 BF16——Top-k 选择器获得 2× 加速，同时 KV 召回率保持 **99.7%**。

3.5 训练框架

训练框架构建在 DeepSeek-V3 的可扩展基础设施之上，针对 V4 的新型组件（Muon, mHC, Hybrid Attention）引入了关键创新。

3.5.1 Muon 的高效实现

核心矛盾：Muon 需要**完整梯度矩阵**来计算更新（Newton-Schulz 需要整个矩阵），而传统 ZeRO 将参数矩阵分片到多个 rank。分片后每个 rank 只有部分梯度，无法独立完成 Newton-Schulz 迭代。

混合 ZeRO 策略：

(a) 密集参数

- **限制最大 ZeRO 并行度**——确保每个 rank 管理足够大的参数矩阵块以完成 Newton-Schulz
- 使用**背包算法**将参数矩阵分配到各 rank，保证负载体量大致均衡
- 各 rank 的 bucket 填充至最大 bucket 大小以支持高效 reduce-scatter
- 填充开销：通常 <10%（每个 rank 管理 ≤5 个参数矩阵时）

当数据并行总规模超过 ZeRO 限制时，在额外的数据并行组上**冗余计算**Muon 更新——用额外计算换取总 bucket 内存降低。

(b) MoE 参数

每个专家独立优化。流程：

1. 将所有层所有专家的 Down Projection 矩阵展平 $\rightarrow$ Up Projection $\rightarrow$ Gate 矩阵
2. 填充展平向量确保可均匀分片而不切割任何逻辑独立矩阵
3. 给定专家数量极大，**不限制 ZeRO 并行度**，填充开销可忽略

自动合并：每个 rank 上，同形状连续参数自动合并——批量执行 Newton-Schulz 迭代以提升硬件利用率。

通信优化：MoE 梯度以 BF16 随机舍入方式同步，通信量减半。为保证数值鲁棒性，用两阶段方法替代传统 reduce-scatter：

1. **all-to-all：**交换各 rank 的局部梯度
2. **本地 FP32 求和：**避免低精度加法器的累积误差

3.5.2 mHC 的成本优化

mHC 增加激活内存消耗和 pipeline 阶段间通信量。优化策略：

1. **融合 kernel：**为 mHC 训练和推理精心设计 fused kernel
2. **选择性重计算：**
 - 重计算大多数层间隐藏状态和所有归一化层输入
 - **避免重计算**计算密集型操作（如 GEMM——重计算 GEMM 的代价接近正向传播）
 - 在节省内存和额外计算开销间取得平衡
3. **DualPipe 1F1B 调整：**修改 DualPipe 的重叠方案，容纳 mHC 增加的 pipeline 通信，使部分 mHC 操作与 GEMM 并发执行

最终开销：仅占重叠 1F1B pipeline 阶段的 **6.7%** wall-time。

3.5.3 上下文并行 (Context Parallelism)

传统 CP 在序列维度分区，每个 rank 维护连续的 s 个 token。压缩注意力引入两个新挑战：

挑战 1：训练样本由多个序列打包，各序列独立压缩（因子 m 或 m' ），不足 m 的尾部 token 被丢弃。各 rank 的压缩长度可能不同。

挑战 2：压缩需要 m 个连续 KV 项，可能跨越相邻 CP rank 边界。

两阶段通信方案：

```
1 Stage 1 (发送边界 tokens):
2   Rank i —[最后m个未压缩KV]—> Rank i+1
3   Rank i+1 压缩: 接收到的 tokens + 本地 tokens → 固定长度 (s/m+1) 压缩项 (含
   填充)
4
5 Stage 2 (全局收集):
6   All-Gather 所有 rank 的局部压缩 KV
7   Fused Select-and-Pad 算子重组全局压缩 KV (总长 = cp_size·s/m)
8   填充项置于尾部
```

对于 HCA 和 CSA 索引器，每个查询 token 的可见压缩 KV 范围可通过规则预计算。对于 CSA 稀疏注意力，top-k 选择器显式指定各查询的可见压缩 KV 索引。

3.5.4 基于 TorchFX 的张量级激活检查点

传统方法的局限：

- 模块级检查点：粗粒度，要么全部保留要么全部重计算，次优的权衡
- 手动实现：细粒度但需显式管理张量状态，大幅增加开发复杂度

V4 方案：基于 TorchFX 的自动微分扩展：

1. 开发者仅需实现前向传播并**选择性标注**个别张量需要检查点/重计算
2. 框架通过 TorchFX 追踪完整计算图
3. 对每个标注张量，**反向遍历**计算图识别其重计算所需的**最小子图**
4. 将最小子图插入反向逻辑——在对应梯度计算之前执行重计算

工程优势：

- 通过**直接释放**标注张量的 GPU 内存并**重用**重计算张量的存储指针——零 GPU 内存拷贝
- 图追踪执行具体模型，可追踪每个张量的底层存储指针
- **自动去重**共享存储的张量（如 reshape 操作的输入/输出），开发者无需关注底层内存细节
- 与手动实现相比**零训练时额外开销**

3.6 推理框架

推理框架继承自 DeepSeek-V3，在 KV 缓存管理上有重大差异。

3.6.1 KV 缓存结构与管理

异构 KV 项的来源:

KV 类型	来源	特殊性
CSA Main KV	CSA 压缩 KV	压缩率 1/4, 共享 KV MQA
CSA Indexer KV	Lightning Indexer	额外维度 ($c^I = 128$), 仅用于索引
HCA KV	HCA 压缩 KV	压缩率 1/128, 共享 KV MQA
SWA KV	滑动窗口分支	未压缩, 仅保留最近 n_{win}
未完成压缩 tokens	压缩缓冲区	等待凑够 m 或 m' 个 tokens

两组件 KV 缓存架构:

(a) State Cache (状态缓存)

将 SWA 和未完成压缩 tokens 视为"序列特定状态"——其内容仅取决于当前位置。预分配固定大小有限的状态缓存池, 动态分配给各序列。

这从根本上绕过了 PagedAttention 对 SWA 和压缩缓冲区的假设违规问题——因为这些缓存具有完全不同的驱逐策略。

(b) Classical KV Cache (经典 KV 缓存)

多个块 per 请求。为支持异构压缩率, 每个缓存块覆盖 $\text{lcm}(m, m') = \text{lcm}(4, 128) = 128$ 个原始 tokens:

- 1 每个块包含:
- 2 $k1 = \text{lcm}(m, m') / m = 128/4 = 32$ 个 CSA 压缩 tokens
- 3 $k2 = \text{lcm}(m, m') / m' = 128/128 = 1$ 个 HCA 压缩 tokens

稀疏注意力 kernel 协同设计:

传统高性能注意力 kernel 假设每块固定 B 个 token。通过高性能稀疏注意力 kernel, 不同层可容纳不同的每块 token 数而不降性能。关键设计——填充块以对齐缓存行 (cache line), 提升内存访问效率。

3.6.2 磁盘 KV 缓存存储

共享前缀请求复用——消除重复 prefilling。

CSA/HCA 缓存: 直接将全部压缩 KV 项存储到磁盘。命中前缀时, 读取并复用直到最后一个完整压缩块的 KV。尾部的未完成压缩块仍需重新计算 (因为未压缩状态不存储)。

SWA KV 缓存: SWA KV 未压缩且存在于每层, 体量约为压缩 KV 的 8 倍。三种策略权衡存储与计算:

策略	存储内容	存储量	恢复计算量	最佳场景
Full SWA Caching	全部 SWA KV	最大	仅读取最后 n_{win} 个	写入成本可忽略的部署
Periodic Checkpointing	每 p tokens 检查点 n_{win} 状态	$1/p$ 的 Full	重算尾部 ($p \bmod$ 命中) tokens	通用, p 可调
Zero SWA Caching	无	零	重算最近 $n_{win} \cdot L$ 层	计算资源充足

Zero SWA Caching 的关键洞察：每层 SWA KV 仅取决于前一层最近 n_{win} tokens。利用已缓存的 CSA/HCA KV，只需重算最后 $n_{win} \cdot L$ 个 token 即可恢复 L 层模型的所有 SWA KV。

Full SWA Caching 在现代 SSD 系统上效率低——因为每次命中仅访问存储中极小部分（最后 n_{win} 个），产生不平衡的以写为主的访问模式。

4. 预训练

4.1 数据构建

- 总规模超过 **32T tokens**
- 数据类别：数学、编程、网页、长文档等
- **新增强化：**过滤批量自动生成/模板化内容，防止模型崩溃
- **中期训练：**加入 agentic 数据提升编程能力
- **长文档优先：**科研论文、技术报告等具有学术价值的材料
- 继承 V3 的文档填充 (Fill-in-Middle)、token 拆分和文档打包策略
- **新特性：**预训练阶段采用样本级注意力掩码

4.2 训练设置

超参数	DeepSeek-V4-Flash	DeepSeek-V4-Pro
训练 tokens 数	32T	33T
最大批次大小	75.5M tokens	94.4M tokens
峰值学习率	2.7×10^{-4}	2.0×10^{-4}
最终学习率	2.7×10^{-5}	2.0×10^{-5}
序列长度演进	4K $\rightarrow$ 16K $\rightarrow$ 64K $\rightarrow$ 1M	4K $\rightarrow$ 16K $\rightarrow$ 64K $\rightarrow$ 1M
密集注意力预热	1T tokens	更长阶段
MTP 损失权重	0.3 $\rightarrow$ 0.1 (LR 衰减时)	0.3 $\rightarrow$ 0.1 (LR 衰减时)

AdamW 配置： $\beta_1 = 0.9$, $\beta_2 = 0.95$, $\epsilon = 10^{-20}$, weight_decay=0.1
Muon 配置：momentum=0.95, weight_decay=0.1, RMS 缩放因子=0.18

4.3 训练稳定性策略

在万亿参数规模训练中遇到的稳定性挑战及解决方案：

Anticipatory Routing (预期路由)

- **问题：**路由机制本身加剧 MoE 层中的异常值，形成恶性循环
- **方案：**将骨干网络更新与路由网络更新解耦
 - 在 step t ，使用当前参数 θ_t 计算特征，使用历史参数 $\theta_{t-\Delta t}$ 计算路由
 - 提前在 step $t - \Delta t$ 预取数据并缓存路由索引
 - 自动检测 loss spike $\rightarrow$ 短暂触发 $\rightarrow$ 恢复正常训练
- **开销：**约 20% wall-time，但仅在异常时自动触发

SwiGLU Clamping

- 将 SwiGLU 的线性分量限制在 $[-10, 10]$
- 将门控分量上限限制为 10
- 有效消除异常值，不损害模型性能

4.4 基座模型评测

Benchmark	V3.2-Base	V4-Flash-Base	V4-Pro-Base
MMLU	87.8	88.7	90.1
MMLU-Pro	65.5	68.3	73.5
SimpleQA (verified)	28.3	30.1	55.2
SuperGPQA	45.0	46.5	53.9
BBH	87.6	86.9	87.5
HumanEval	62.8	69.5	76.8
MATH	60.5	57.4	64.5
LongBench-V2	40.2	44.7	51.5

关键发现：

- V4-Flash 以不到 V3.2 一半的激活参数量，在多数基准上超越 V3.2
 - V4-Pro 实现了全面领先，尤其在知识密集型基准上提升显著
-

5. 后训练

5.1 后训练流水线

采用两阶段范式：领域专家独立训练 → On-Policy Distillation (OPD) 统一整合

5.1.1 专家训练

- 各目标领域（数学、编程、Agent、指令遵循等）独立训练
- 流程：SFT → GRPO (RL)
- 生成式奖励模型 (GRM)：用于难以验证的任务，Actor 网络本身作为 GRM，联合优化生成和评判能力

5.1.2 三种推理模式

模式	特点	典型场景
Non-think	快速直觉响应	日常任务、低风险决策
Think High	有意识逻辑分析	复杂问题求解
Think Max	推理推至极限	探索模型推理能力边界

Think Max 模式注入的系统指令：

"Reasoning Effort: Absolute maximum with no shortcuts permitted. You MUST be very thorough..."

5.1.3 工具调用 Schema

- 引入 `|DSML|` 特殊 token 和 XML 格式工具调用
- `<think></think>` 标签界定推理路径
- XML 格式有效减少转义失败和工具调用错误

5.1.4 Interleaved Thinking

- 工具调用场景：完整保留所有推理历史（含跨用户消息边界），支持长时程 Agent 任务
- 通用对话场景：新用户消息到达时丢弃前轮推理内容，保持上下文简洁

5.1.5 Quick Instruction

- 在输入序列后附加专用特殊 token 处理辅助任务（搜索触发、意图识别、标题生成等）
- 复用已有 KV 缓存，零额外 prefilling
- 显著降低首 token 延迟 (TTFT)，消除维护额外小模型的工程负担

5.2 On-Policy Distillation (OPD)

目标函数:

$$\mathcal{L}_{OPD}(\theta) = \sum_{i=1}^N w_i \cdot D_{KL}(\pi_{\theta} \| \pi_{E_i})$$

- 多个领域专家教师 → 单个统一学生模型
- 使用完整词表 **logit 蒸馏** 而非 token-level KL 估计
- 超过 10 个教师模型覆盖各领域

5.3 RL/OPD 基础设施

5.3.1 FP4 量化集成

- Rollout 和教师/参考模型推理: 直接使用真实 FP4 权重
- 训练步骤: 通过无损 FP4→FP8 反量化模拟, 无缝复用 FP8 混合精度框架

5.3.2 高效教师调度

- 教师权重卸载到集中式分布式存储, 按需加载
- 仅缓存教师最后一层隐藏状态 (而非完整 logits), 训练时动态重建
- 按教师索引排序训练样本, 确保每个 mini-batch 至多一个教师头驻留 GPU
- 专用 TileLang kernel 加速 KL 散度计算

5.3.3 可抢占和容错 Rollout 服务

- **Token 粒度的 Write-Ahead Log (WAL)**: 每生成一个 token 立即持久化
- 抢占时保存未完成请求的 KV 缓存
- 恢复时使用 WAL + KV 缓存继续解码
- **正确性关键**: 从头重新生成会引入长度偏差 (短响应更容易在中断前完成)

5.3.4 百万 Token 上下文 RL

- 轻量级元数据 + 重量级逐 token 字段的分解数据格式
- 共享内存数据加载器消除节点内数据冗余
- 动态 mini-batch 数量, 平衡计算吞吐与 I/O

5.3.5 Agent Sandbox 平台: DSec

基于 Rust 的三组件架构 (Apiserver, Edge, Watcher), 核心能力:

- **四种执行基座**: Function Call / Container / microVM (Firecracker) / fullVM (QEMU), 统一接口
- **分层存储快加载**: EROFS 叠加层 + overlaybd, 按需加载

- 高密度并发：同一集群管理数十万并发沙箱实例
- 轨迹日志：全局有序记录，支持快进恢复和确定性回放

6. 评测结果

6.1 知识与推理

Benchmark	DS-V4-Flash-Max	DS-V4-Pro-Max	GPT-5.4-xHigh	Gemini-3.1-Pro-High
MMLU-Pro	89.1	90.2	87.5	91.0
SimpleQA-Verified	46.2	57.9	45.3	75.6
Chinese-SimpleQA	76.4	84.4	76.8	85.9
GPQA Diamond	91.3	90.1	93.0	94.3
HLE	40.0	37.7	39.8	44.4
Codeforces (Rating)	3168	3206	3168	3052
Apex	85.9	90.2	78.1	89.1

6.2 Agent 性能

Benchmark	DS-V4-Pro-Max	Claude-Opus-4.6-Max	GPT-5.4-xHigh
Terminal Bench 2.0	67.9	65.4	75.1
SWE Verified	80.6	80.8	—
SWE Pro	55.4	57.3	57.7
SWE Multilingual	76.2	77.5	82.7
BrowseComp	83.4	83.7	—
Toolathlon	51.8	47.2	54.6

6.3 真实场景性能

6.3.1 中文写作

对比	DS-V4-Pro 胜率	对手胜率	平局
vs Gemini-3.1-Pro（功能写作）	62.7%	34.1%	3.3%
vs Gemini-3.1-Pro（创意写作-写作质量）	77.5%	22.4%	0.2%
vs Claude-Opus-4.5（复杂约束/多轮）	45.9%	52.0%	2.0%

6.3.2 研发编程

模型	通过率
Claude-Haiku-4.5	13%
Claude-Sonnet-4.5	47%
DeepSeek-V4-Pro-Max	67%
Claude-Opus-4.5	70%
Claude-Opus-4.5 Thinking	73%
Claude-Opus-4.6 Thinking	80%

内部开发者调查 (N=85):

- 52% 认为可作为主力编程模型
- 39% 倾向于认同
- <9% 不认可
- 主要不足：偶尔出现低级错误、对模糊 prompt 误解、偶尔过度思考

6.3.3 1M 长上下文

Benchmark	DS-V4-Pro-Max	Claude-Opus-4.6	Gemini-3.1-Pro
MRCR 1M (MMR)	92.9	94.9	76.3
CorpusQA 1M (ACC)	71.7	87.0	53.8

MRCR 检索性能在 128K 内高度稳定，超出后虽有下降但在 1M 仍非常强劲。

6.4 推理努力与效率

DeepSeek-V4 在增加测试时计算量时，实现了相比 DeepSeek-V3.2 更高的 token 效率。在 HLE 任务上，V4-Pro 以更少的总 token 数达到更高 Pass@1。

6.5 Search 增强问答

RAG 搜索 (V4-Pro vs V3.2)

类别	V4 胜率	V3.2 胜率	平局
客观问答	27.3%	8.7%	64.0%
主观问答	28.5%	11.1%	60.4%
整体	28.1%	10.4%	61.5%

Agentic Search vs RAG

- Agentic Search 在复杂任务上显著优于 RAG
- 成本仅略高于 RAG (平均 16.2 次工具调用 vs 0)

7. 工程实践亮点

7.1 软硬件协同设计

论文提出了多项对硬件厂商的建议：

1. **计算-通信比率**：一旦带宽满足 $C/B \leq 2d$ 阈值，增加带宽收益递减
2. **功耗预算**：极端 kernel 融合同时驱动计算、内存、网络高负载，需要足够功耗裕量
3. **通信原语**：采用 pull-based 设计（GPU 主动读取远程数据），建议未来硬件支持更低延迟的跨 GPU 信令
4. **激活函数**：建议用低成本的逐元素激活替代 SwiGLU，去除指数/除法操作

7.2 数值精度保证

- 端到端位精确批次不变性和确定性
- 预训练/后训练/推理流水线间位对齐
- 这对调试、稳定性分析和一致性后训练行为至关重要

7.3 开源

- 模型检查点：<https://huggingface.co/collections/deepseek-ai/deepseek-v4>
 - CSA/HCA 开源实现：<https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/tree/main/inference>
 - MegaMoE mega-kernel：<https://github.com/deepseek-ai/DeepGEMM/pull/304>
-

8. 局限性与未来方向

8.1 架构复杂性

- 为了最小化风险，保留了大量初步验证的组件和技巧
- 未来方向：更全面和原则性的研究，蒸馏出最本质的设计，使架构更优雅

8.2 训练稳定性理解不足

- Anticipatory Routing 和 SwiGLU Clamping 虽然有效，但其底层原理尚不完全清楚
- 未来方向：积极研究训练稳定性基础问题，加强内部指标监控

8.3 未来探索方向

- 模型稀疏性**: 探索更稀疏的 Embedding 模块等新维度
- 低延迟架构**: 使长上下文部署和交互更快响应
- 长时程多轮 Agent 任务**: 持续迭代改进
- 多模态能力**: 正在整合中
- 更好的数据策略**: 持续提升模型智能、鲁棒性和实用可用性
- 在线学习**: 基于百万 token 高效上下文的新范式探索

9. 总结与展望

DeepSeek-V4 代表了开源大语言模型在以下三个维度的重要突破:

核心成就

- 效率革命**: 通过 CSA/HCA 混合注意力, 在 1M-token 场景下仅需 V3.2 的 10-27% FLOPs 和 7-10% KV 缓存
- 能力跃升**: DeepSeek-V4-Pro-Max 在知识、推理、Agent、长上下文等维度全面达到开源 SOTA, 在多个指标上接近甚至匹敌最先进的闭源模型
- 工程创新**: mHC、Muon、FP4 QAT、MegaMoE、TileLang、DSec 等一整套系统级创新

差距与定位

- 与 GPT-5.4 / Gemini-3.1-Pro 等前沿闭源模型的差距约 **3-6 个月**
- 在知识型任务上仍落后于 Gemini-3.1-Pro
- 在 Agent 任务上与 Kimi-K2.6 和 GLM-5.1 等领先开源模型持平

里程碑意义

DeepSeek-V4 系列开启了**开源模型百万 token 上下文时代**, 为测试时扩展、长时程任务和在线学习等未来范式奠定了必要基础。其效率突破使得百万 token 上下文从理论上的可行变为工程上的实用, 这可能是比纯粹的能力提升更具深远意义的技术贡献。

参考文献: DeepSeek-AI. "DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence." 2026.

报告生成日期: 2026-04-26